

Gerenciamento da construção e liberação

Você vai compreender os fundamentos do gerenciamento de construção e de liberação, explorando boas práticas e ferramentas como Maven, Gradle e Jenkins, essenciais para a automação e controle de dependências e versões em projetos de software.

Rodrigo Augusto dos Santos

1. Itens iniciais

Propósito

O conhecimento sobre gerenciamento de construção e de liberação é essencial para profissionais de TI automatizarem processos, controlarem versões e gerirem dependências de forma eficiente, garantindo entregas de software mais seguras, estáveis e alinhadas às práticas de integração e entrega contínua.

Objetivos

- Analisar os fundamentos do gerenciamento de construção, incluindo boas práticas e ferramentas como Maven e Gradle, com foco na gestão de dependências.

- Avaliar os fundamentos, tipos e boas práticas do gerenciamento de liberação, reconhecendo o uso da ferramenta Jenkins nesse contexto.

Introdução

Neste conteúdo, voltaremos nossa atenção para aspectos fundamentais do ciclo de vida de desenvolvimento de software, com foco no

Gerenciamento de construção e no gerenciamento de liberação. Esses processos são essenciais para garantir a entrega eficiente, automatizada e confiável de aplicações, desde a sua construção até a disponibilização final ao usuário.

Começaremos explorando os principais conceitos do gerenciamento de construção, destacando seus fundamentos e práticas recomendadas, com ênfase na gestão de dependências. A gestão adequada dessa etapa permite que os sistemas sejam construídos de maneira consistente, evitando conflitos entre bibliotecas e promovendo maior controle sobre o ambiente de desenvolvimento.

Em seguida, conheceremos duas das ferramentas mais populares e amplamente utilizadas nessa área: Maven e Gradle. Ambas exercem papel central na automação da construção de projetos, oferecendo recursos para configuração, compilação, empacotamento e até mesmo execução de testes automatizados.

Na sequência, abordaremos um tópico essencial dentro da gerência de configuração: o gerenciamento de liberação. Serão apresentados seus principais conceitos, os diferentes tipos de liberação e um conjunto de boas práticas que ajudam a estruturar processos mais previsíveis e seguros para a entrega de versões de software.

Por fim, será apresentada uma visão geral sobre o uso do Jenkins, uma ferramenta consagrada no mercado, especialmente voltada para o suporte ao gerenciamento de liberação. Sua aplicação prática permite a orquestração de tarefas de integração e entrega contínuas, promovendo maior agilidade e qualidade nas entregas.

Ao final deste conteúdo, você terá uma visão abrangente e aplicada das práticas e ferramentas que fortalecem a automação e o controle nos processos de construção e liberação de software.

A gestão de construção e seus fundamentos

Vimos anteriormente que o controle de mudanças usa um volume considerável de automação para ser executado. Tal automação é fundamental para que o controle de mudanças seja bem executado, afinal, o volume de itens de configuração de software sob controle somado ao já esperado e normalmente alto volume de mudanças a todo tempo fazem com que uma abordagem manual seja impraticável, senão impossível.

Essa necessidade por automação não é algo inerente somente ao controle de mudanças, mas será um aspecto doravante corriqueiro, inerente a todos os tópicos que visitaremos, a ponto de ficar claro que a gerência de configuração como um todo não ocorre sem forte uso de ferramentas automatizadas. O primeiro dos demais tópicos que se encaixa neste paradigma é a gestão de construção, já que este assunto representa uma parte importante dos ciclos de vida de desenvolvimento e teste.



Atenção

A gestão de construção envolve justamente o passo de coletar todos os ativos e itens de configuração a serem incluídos e uni-los em uma construção (build), seja como parte de uma nova implementação, devido à correção de defeitos, em resposta a mudanças, em preparação para atender a uma liberação, concretizar uma implantação, e ainda por quaisquer outras razões pertinentes.

A palavra gestão diz justamente respeito a automatizar e orquestrar de forma ativa tudo o que é necessário, dando um caráter mais profissional ao que eram tarefas normais do dia-a-dia.

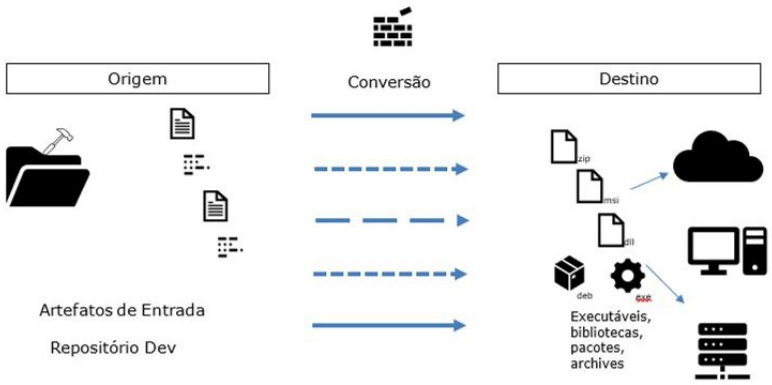
> Em outras palavras, a gestão de construção é toda a orquestração da automação necessária para que de fato se gere um produto de software concreto. Estão incluídas aqui as tarefas para compilar, construir e empacotar o produto, e possivelmente ainda testá-lo e implantá-lo em ambientes de teste diferentes.

Por mais que a gerência de construção seja automatizada, veremos mais adiante que esses passos na verdade são apenas iniciais na caminhada rumo à visão avançada promovida pela integração contínua e DevOps.



Conteúdo interativo

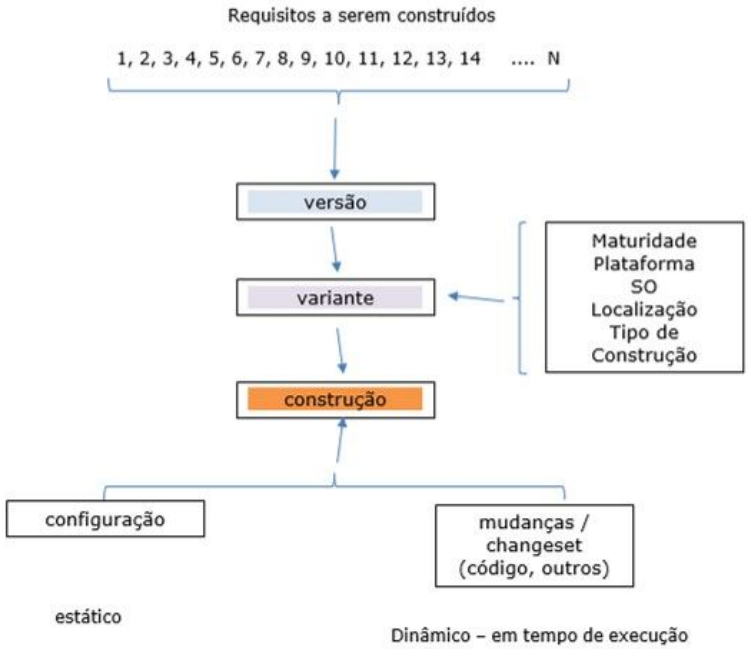
Acesse a versão digital para ver mais detalhes da imagem abaixo.



Visão da gestão de construção.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



A ligação entre construção e outros conceitos de configuração.

As construções são, portanto, a concretização da adição de funcionalidades e / ou modificações requeridas para que se atinjam as performances esperadas em termos de requisitos contratados, sejam esses requisitos funcionais ou não funcionais.

À medida que as aplicações crescem, é impossível ignorar a complexidade aumentada que elas trazem consigo; logo, atividades normais do dia a dia passam a requerer gestão, sendo a construção apenas uma dessas atividades.

O gerenciamento de dependências

O gerenciamento de dependências é um assunto diretamente ligado ao gerenciamento de construções, e extremamente importante de ser explicitamente abordado. Isso ocorre porque o *software*, de maneira geral, não funciona isoladamente, como um bloco monolítico.

O *software* atual consiste de funcionalidades reutilizáveis que são decompostas em bibliotecas ou componentes que, ao se relacionarem, compõem juntos alguma forma de estrutura, como, por exemplo, um módulo. A estes relacionamentos entre componentes damos o nome de *dependências*.



Atenção

O gerenciamento de dependências envolve a declaração, resolução e uso dessas dependências de maneira automatizada, ou seja, diretamente viabiliza as construções para os seus vários propósitos. Basicamente permitem reuso de funcionalidades, algo muito importante atualmente, permitindo ainda a criação de frameworks de desenvolvimento de software na forma de bibliotecas, componentes de terceiros etc., aumentando assim a escala e a velocidade do desenvolvimento.

O gerenciamento de dependências, portanto, se refere à gestão ativa das várias dependências encontradas em um esforço de desenvolvimento de software, tais como as que são encontradas entre:

- Código-fonte;
- Arquivos individuais;
- Grupos de arquivos ou pacotes de arquivos;
- Bibliotecas internas à organização;
- Bibliotecas externas à organização (publicadas na web, por exemplo);
- Projetos inteiros (jars, por exemplo);
- Outros itens de configuração.

Constatamos aqui a importância da gerência de dependências como prática isolada, bem como em relação ao gerenciamento de construção. Projetos envolvendo dezenas, centenas ou até milhares de dependências cairiam em uma verdadeira espiral negativa caso essas dependências precisassem ser geridas manualmente.

O gerenciamento de dependências automatiza boa parte desse processo, permitindo visualização, navegação e análise das dependências. Isso é ainda mais importante dado o uso cada vez maior de código *open source*.

Dependências são classificadas em dois tipos em particular:

Dependências diretas

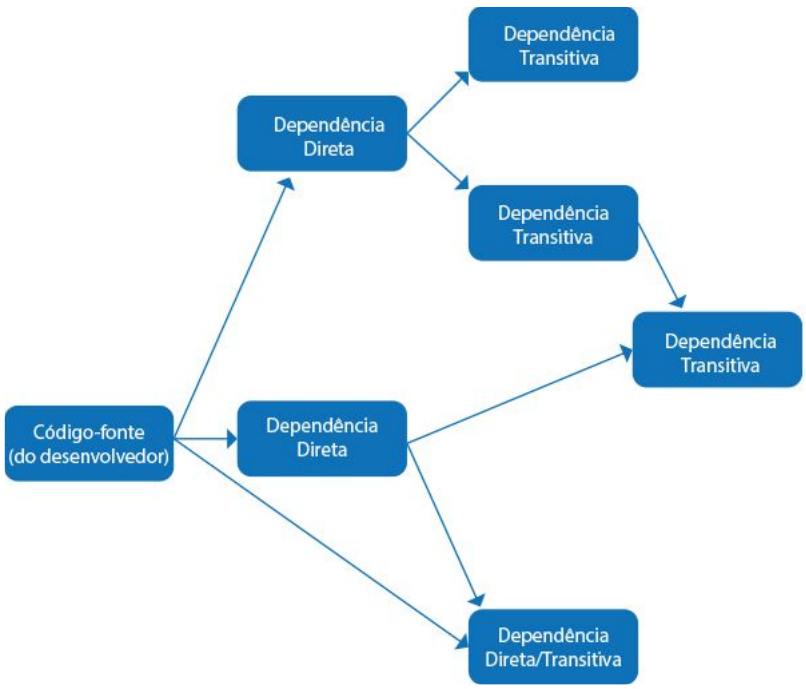
Coisas usadas diretamente pelo desenvolvedor do código-fonte.

Dependências transitivas

Coisas usadas pelas coisas que o desenvolvedor do código-fonte usa, ou seja, coisas usadas recursivamente.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



A diferença entre dependências diretas e transitivas.

Em termos de benefícios tangíveis do gerenciamento de dependências, podemos citar:

Melhor segurança

Componentes ou bibliotecas podem possuir vulnerabilidades. O gerenciamento de dependências ajuda a manter esses componentes atualizados, reduzindo as chances de a aplicação ser exposta e propagar o risco associado a quem a usa.

Melhorias de performance

Se as dependências estão desatualizadas, é provável que não se tenha acesso às últimas melhorias implementadas. A gerência de dependências ajuda diretamente a garantir acesso às últimas atualizações, o que possivelmente afetará a performance do aplicativo de maneira positiva.

Garantia da qualidade

A gestão de dependências ajuda na prevenção de referências circulares ou conflitantes.

Melhor aderência a políticas de licenciamento

Não é somente software proprietário que envolve termos de licenciamento. Software open source também pode trazer consigo termos semelhantes, e o gerenciamento de dependências ajuda a manter em cheque problemas relacionados à propriedade intelectual.

Assista o vídeo e saiba mais sobre a gestão de mudanças.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Boas práticas na gestão de construção

O gerenciamento de construção, embora específico e trazendo consigo méritos próprios, é comumente menosprezado, pois toda organização acaba por realizá-lo em algum nível, quer isto seja reconhecido como uma atividade, um processo ou qualquer outra coisa. Considerando que o foco aqui é a profissionalização desse esforço, comecemos a rever as boas práticas em relação à gestão de construção, com especial atenção a duas práticas específicas.

A primeira delas diz respeito à própria adoção da gerência de construção, o que passa por um bom mapeamento do que precisa ser feito / atingido. Em um primeiro momento, isso pode minimamente incluir coisas como:

- Quais subtarefas precisam estar envolvidas nas construções?
- Quais tipos de construção serão necessários?
- Quais ferramentas de construção são adequadas?

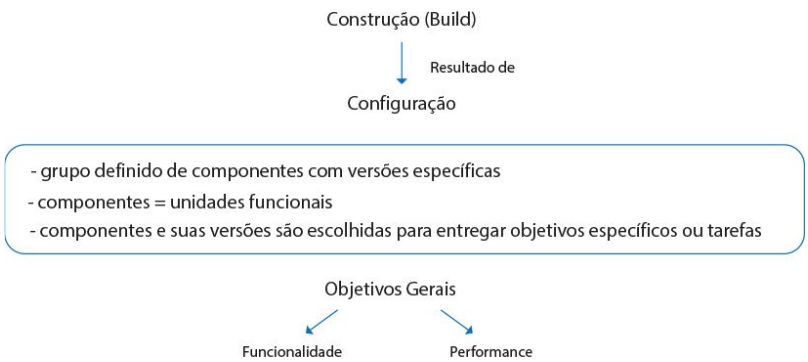
Conforme o tema se aprofunde, e a organização e os profissionais adquiram visibilidade sobre a gerência de construção, novas considerações precisam ser feitas, como por exemplo:

- Quem irá realizar a construção?

- O que será construído?
- Quando ocorrerá a construção?
- Onde ocorrerá a construção?
- Como a construção será completada?
- Quem é responsável pelo sucesso final da construção?
- Quem é a pessoa principal a ser contatada em caso de erros?
- Quais são as dependências de cada construção?
- Quando as construções serão implantadas?
- Em que ambientes cada construção será implantada?

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Razão de ser da gestão de construção.

Um destes pontos para o qual devemos chamar mais atenção é a questão de quem realizará as construções. Uma das abordagens adotadas por organizações é delegar essa função aos analistas de configuração ou mesmo aos gerentes de configuração. Em geral, isso dependerá da carga de trabalho desses profissionais,

independentemente de eles cuidarem da execução das construções; o que é importante ressaltar aqui é a necessidade de segregação de funções.

Isso significa que, qualquer que seja o papel que realize a codificação, ele deve ser separado do que irá realizar a construção. No mínimo, deve haver separação para geração dos *builds* que serão implantados em ambientes de teste e produção.

Isso representa uma salvaguarda para limitar os riscos no código sendo alterado e chegando aos ambientes sem passar pelo processo normal de *check-in*, o que poderia resultar em danos ao ambiente e em efeitos negativos para os usuários e clientes.

É óbvio que em caso de extrema necessidade, como por exemplo uma mudança ou liberação emergenciais, esse racional poderá ser relaxado até certo ponto; no entanto, a gestão de construção deve retornar ao seu estado normal após a situação de crise ter sido contornada, até para garantir que nenhuma área seja afetada pela construção / mudança / liberação emergenciais.

O segundo ponto a ser levado em consideração é a necessidade de sempre se garantir que seja possível criar construções de maneira manual. Isso pode soar contraintuitivo, mas não é, afinal sempre existe a possibilidade de algo dar errado com uma construção automatizada. Um exemplo é o *turnover*, quando os automatizadores de alguma construção em específico deixam a organização.

Outras razões para disrupção na automação de construções podem ser mudanças em sistemas operacionais, um *patch* ou atualização que quebram o processo de construção automatizada. Essas situações não são tão incomuns, e quando ocorrem podem levar dias ou semanas para serem reparadas. Em geral, é inaceitável que se perca a capacidade de realizar construções por tão longo período; logo, a manutenção da habilidade em se realizar construções manuais é altamente justificável.

Tendo sido cobertas essas duas boas práticas de maneira mais aprofundada, vejamos algumas outras boas práticas aplicáveis ao gerenciamento de construção:

- Realize a construção de produtos que estão sendo alterados ao menos uma vez por semana (se possível diariamente);
- Procure otimizar a árvore do código fonte, mais especificamente os ramos da árvore, de forma que para cada componente do produto se tenha no máximo um ramo;
- Comece construções durante o dia, não durante a noite;
- Publique o cronograma de construções oficial em algum lugar, se possível em um website amplamente acessível a todos os envolvidos;
- Adote um padrão claro e consistente para o nome das construções;

- Caso as construções em questão se destinem a uso internacional, se certifique de ter suporte multilíngue;

- Institua check-ins atômicos dos repositórios privados para o principal, ou seja, ao se realizar check-ins, todos os artefatos / itens de configuração devem fazer parte dele. Não se deve deixar nenhum fora.

Assista o vídeo e aprenda as boas práticas sobre a gerência de configuração.

> Conteúdo interativo
>
> Acesse a versão digital para assistir ao vídeo.

Identificar e conhecer a ferramenta Maven

A ferramenta Maven é uma popular ferramenta, idealizada em 2002, para ser empregada em gestão de projetos. Apesar disso, o Maven realmente se tornou popular por suas capacidades de automação de construção, sendo usado principalmente em projetos de software que fazem uso da linguagem de programação Java. O Maven, no entanto, é capaz de suportar múltiplas linguagens de programação, entre elas Ruby on Rails, C, C# e outras.

A ferramenta Maven se estrutura ao redor do conceito de Project Object Model (POM), que é implementado por um arquivo XML, descritivo do projeto de software que está sendo construído. Essa descrição, embutida no arquivo POM, inclui coisas como:

- Projeto sendo construído;

- Dependências;

- Ordem de construção;

- Diretórios;

- Plug-ins.

O pom.xml pode prover a configuração para um projeto inteiro, mas é comum que, à medida que os projetos cresçam em tamanho e complexidade, eles sejam divididos em módulos, cada um com seu próprio POM se reportando a um POM geral-raiz.

Também é importante dizer que é possível manter relacionamentos pai-filhos entre os POMs, ou seja, o POM filho herda todos os elementos de configuração do pai. O POM raiz também pode ser o super POM, que é geral, fornecido pelo próprio Maven, e que se sobrepõe a outros POMs.

Comentário

A estrutura POM, ao mesmo tempo em que traz simplicidades e facilidades, como, por exemplo, a centralização da configuração, também é apontada como fonte de problemas, pois, à medida em que os projetos crescem, a estrutura XML passa a ficar cada vez mais desorganizada.

Outro ponto é que o tempo de construção tende a apresentar menos previsibilidade, apresentando cada vez maiores taxas de variação.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

pom.xml (BookStore)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example.maven</groupId>
    <artifactId>BookStore</artifactId>
    <packaging>pom</packaging>
    <version>1.0-SNAPSHOT</version>
    <modules...>
     <profiles>
         <profile...>
         <profile>
             <id>productionServer</id>
             <properties>
                 <database.url>
                     jdbc:postgresql://host/database
                 </database.url>
             </properties>
             <dependencies>
                 <dependency>
                     <groupId>org.postgresql</groupId>
                     <artifactId>postgresql</artifactId>
                     <version>9.4-1206-jdbc4</version>
                 </dependency>
             </dependencies>
         </profile>
     </profiles>
    <dependencies...>
</project>
```

Visão interna de um arquivo pom.xml.

Identificar e conhecer a ferramenta Gradle

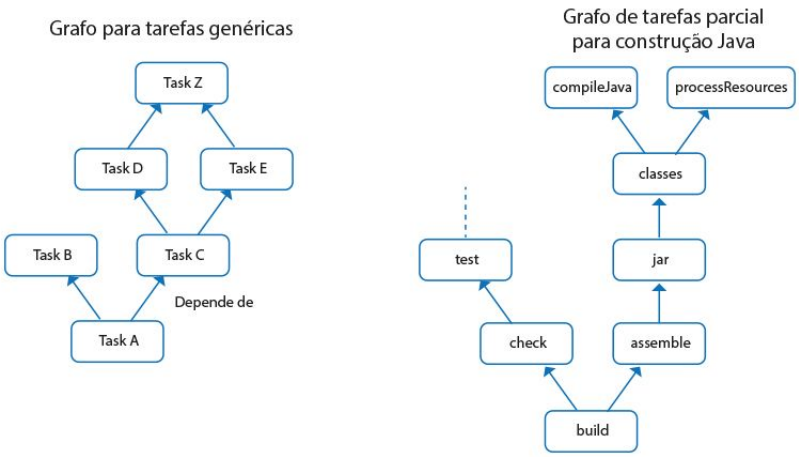
O Gradle é outra ferramenta voltada para o suporte da automação em construção. Famosa por ser empregada no Android Studio, onde constrói apps móveis para a plataforma Android, é flexível o bastante para ser usada em conjunto com outras plataformas e/ou linguagens de programação, tais como Python, por exemplo. Essa flexibilidade se dá em parte graças ao fato de o Gradle rodar sobre uma Java Virtual Machine.

Saiba mais

Outra parte da flexibilidade (e agilidade) do Gradle vem de sua concepção baseada no emprego de Grados Diretos Acíclicos para as tarefas. Este grafo é baseado nas dependências apresentadas entre as tarefas de construção, e após gerado torna mais fácil para a ferramenta determinar a melhor ordem para execução das tarefas, o que ocorre após essa determinação. Essa é uma maneira bem agnóstica de representar dependências e as sequências envolvidas na construção.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Em geral, as maiores reclamações a respeito do Gradle giram ao redor da necessidade em se ter maiores habilidades técnicas para se conseguir operá-lo. Outro ponto é que a comunidade que suporta o Maven é maior do que a que suporta o Gradle. Assim, desenvolvedores usando Gradle tendem a levar mais tempo para conseguirem resolução de dúvidas e problemas.

Assista o vídeo e conheça um pouco mais sobre as ferramentas de Maven e Gradle.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Verificando o aprendizado

Questão 1

A gestão de construção não inclui:

A Coleta de itens de configuração.

B Compilação.

C União de ativos.

D Orquestração.

E Implantação.

A alternativa E está correta.

A gestão de construção envolve justamente o passo de coletar todos os ativos e itens de configuração a serem incluídos e uni-los em uma construção (build), seja como parte de uma nova implementação, devido à correção de defeitos, em resposta a mudanças, em preparação para atender a uma liberação, concretizar uma implantação, e ainda por quaisquer outras razões pertinentes.

A palavra gestão diz respeito a automatizar e orquestrar de forma ativa tudo o que é necessário para a construção, dando um caráter mais profissional ao que eram tarefas normais do dia-a-dia.

Questão 2

Não é uma vantagem advinda do uso de dependências:

A Melhor performance.

B Reuso.

C Criação de frameworks.

D Maior segurança.

E Redução de defeitos.

A alternativa E está correta.

Redução de defeitos não pode ser diretamente atribuída ao uso de dependências.

Questão 3

A automação proporcionada pela gestão da construção:

A Não tem a capacidade de substituir plenamente a construção manual.

B Tem capacidade de substituir plenamente a construção manual.

C Deve mandatoriamente substituir plenamente a construção manual.

D Não deve jamais substituir plenamente a construção manual.

E Não tem ligação com a construção manual.

A alternativa A está correta.

Sempre existe possibilidade de algo dar errado com uma construção automatizada. Um exemplo é o turnover, quando os automatizadores de alguma construção em específico deixam a organização.

Outras razões para disrupção na automação de construções podem passar por mudanças em sistemas operacionais, um patch ou atualização que quebram o processo de construção automatizada.

Essas situações não são tão incomuns, e quando ocorrem podem levar dias ou semanas para serem reparadas. Em geral, é inaceitável que se perca a capacidade de realizar construções por tão longo período; logo, a manutenção da habilidade de se realizar construções manuais é altamente justificável.

Questão 4

A ferramenta Gradle funciona:

A Com base no Linux.

B Rodando sobre uma Sandbox.

C Rodando sobre uma JVM.

D Com base em Python.

E Com base em C#.

A alternativa C está correta.

O Graddle tem alta flexibilidade, e isso se dá em parte graças ao fato de ele rodar sobre uma Java Virtual Machine.

O gerenciamento de liberação e seus fundamentos

Vimos que itens de configuração são fundamentais como elementos habilitadores de projetos, operações e outros empreendimentos, daí a necessidade da gestão cuidadosa de suas configurações. Vimos também que itens de configuração novos ou alterados podem ser resultado direto ou produto desses empreendimentos.



Comentário

Independentemente do papel que tenham, é normal que, ao longo de seus ciclos de vida, itens de configuração passem por muitas alterações, necessitando, portanto, do controle de mudanças, de forma que se mantenha níveis aceitáveis de rastreabilidade e supervisão sobre cada mudança solicitada.

O controle de mudança cuida do aceite (ou rejeite) e da implementação da mudança em si, mas é necessário algo a mais para que essa mudança chegue ao ambiente operacional, ou seja, o ambiente disponibilizado aos usuários, sejam estes usuários internos ou externos à organização.

O gerenciamento de liberação é esse algo a mais, e tem por propósito assegurar a distribuição (ou liberação) de itens de configuração para fora das fronteiras das atividades de desenvolvimento.

> Estão inclusos no gerenciamento de liberação a identificação, empacotamento e entrega direta dos elementos componentes de um produto, como, por exemplo, os arquivos executáveis, documentação, manual de usuário, notas de liberação e dados de configuração.

É importante mencionar que existe uma íntima ligação entre liberação e testes, sejam eles de verificação (que focam em defeitos - conduzidos por equipes de teste e / ou qualidade) ou de validação (que focam em conformidade com os requisitos - conduzidos por usuários ou clientes).

Procedimentos de teste podem estar diretamente acoplados aos de liberação, ou podem ser representados por procedimentos totalmente independentes. O nosso foco é na liberação em si, portanto não daremos enfoque aos testes.

Considerando que mudanças basicamente ocorrem a todo momento, fica óbvio que não é prático gerar uma liberação correspondente para toda e cada mudança. Dessa maneira, uma preocupação do gerenciamento de liberação é determinar quando são justificáveis a formação e emissão de uma liberação.

Exemplo

Coisas que vão desde a severidade dos problemas sendo endereçados, até mensurações operacionais que apontem para uma alta densidade de defeitos originários de liberações anteriores podem ser elementos que afetam a decisão.

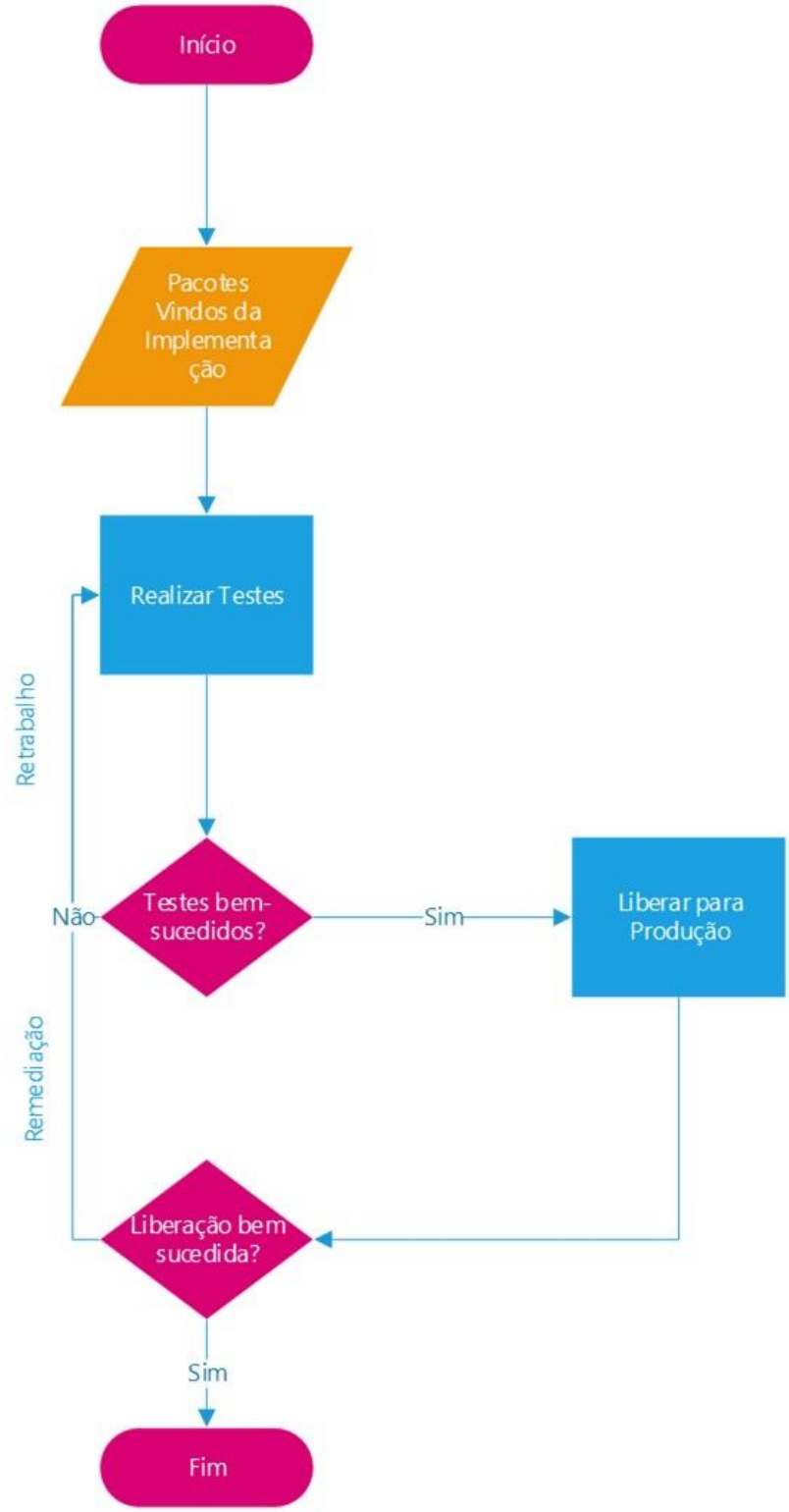
Uma vez que seja decidido que uma liberação é necessária, o empacotamento deve identificar claramente que itens de configuração componentes do produto sendo liberado devem ser entregues, e então deve selecionar as variantes corretas desses itens de configuração.

Por mais que os itens de configuração e a liberação em si sejam compostos por elementos intangíveis, pois se trata afinal de *software* e/ou documentação, pode-se dizer que dispõem de informação física, documentada em um **documento de descrição de versão**.

As notas de liberação normalmente irão descrever as novas capacidades, problemas conhecidos e os requisitos de plataforma necessários para a operação adequada do produto. O pacote a ser liberado também pode conter instruções para instalação e/ou atualização.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Possível fluxos integrados de liberação e teste.

As informações de atualização podem ser mais complicadas de serem adquiridas e formalizadas, pois pode ser o caso de que alguns usuários podem estar usando versões que foram liberadas há muito tempo, estando, portanto, bem desatualizadas.

Esse é inclusive um motivo pelo qual é possível que seja delegada ao gerenciamento da liberação a responsabilidade por manter a rastreabilidade da distribuição do produto em uso pelos diferentes usuários. Outro ponto importante, e que também pode ser usado para esse propósito, é a implementação de algum

mecanismo que assegure a integridade do item liberado, como por exemplo algum tipo de assinatura digital, podendo assim ter o propósito duplo de servir como incentivo para adoção das novas versões sendo liberadas.

Os diferentes tipos de liberação

Vimos que convém definir critérios claros e objetivos para que liberações sejam emitidas, já que existe uma forte ligação entre os dois conceitos. A principal razão das liberações existirem é o de transicionar mudanças para o ambiente operacional, e não existe uma relação 1 para 1 entre eles, além do fato de que existem tipos diferenciados de mudanças, bem como tipos diferenciados de liberação.

Novamente, como fizemos no caso do controle de mudanças, para o gerenciamento de liberação iremos recorrer à *Information Technology Infrastructure Library* (ITIL) para definir diferentes tipos de liberação.

Liberações baseadas no nível de impacto

Listaremos três tipos, e é importante ressaltar que cada um deles pode implementar de uma a n mudanças por vez.

Liberações maiores

São liberações que têm como objetivo primário o de entregar novas funcionalidades, em geral, de amplo escopo. São exemplos desse tipo de liberação grupos de telas ou relatórios, novos módulos, novas integrações etc. Como objetivo secundário, podem corrigir problemas, inclusive eliminando soluções temporárias que foram introduzidas via liberações menores ou mesmo emergenciais.

Se isso ocorrer, a liberação maior substitui e elimina as liberações menores e emergenciais que a precederam. É comum que toda liberação, ao ser executada, gere a atualização da numeração da versão de um item de configuração.

No caso das liberações maiores, pode-se definir uma forma específica de avanço de numeração versão, de forma a diferenciar a liberação maior das demais. Um exemplo possível pode ser avançar um incremento completo, como versão 2.40 para versão 3.0.

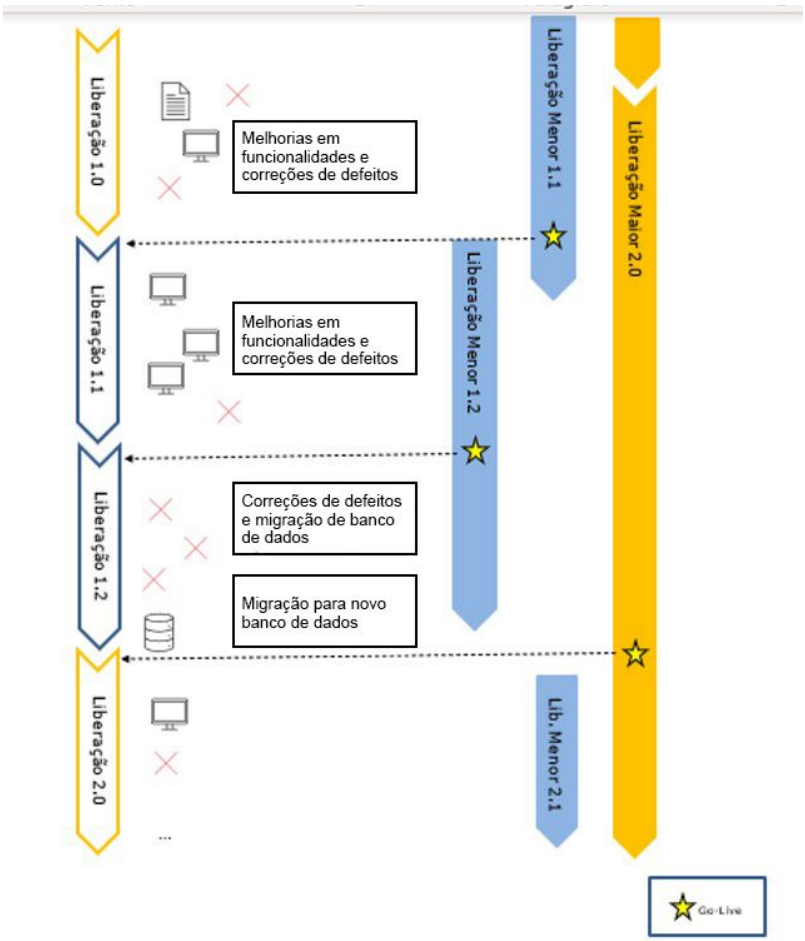
Liberações menores

São liberações que têm como objetivo primário o de implementar correções para problemas existentes em uma linha de base ou estado acordado de um item de configuração. Liberações menores não tendem a adicionar novas funcionalidades, mas, se o fizerem, tal funcionalidade nunca será ampla, sendo restrita a pequenas melhorias, tais como uma nova tela, um novo relatório etc.

Essas liberações devem incrementar o número de versão de maneira diferente das liberações maiores. Um exemplo poderia ser 2.40 para 2.50.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Possível fluxo para mudança normal.

Liberações emergenciais

Liberações emergenciais normalmente se destinam a implementar mudanças emergenciais, ou seja, endereçam consertos rápidos para problemas inesperados ou soluções preventivas para problemas que estão na iminência de ocorrer.

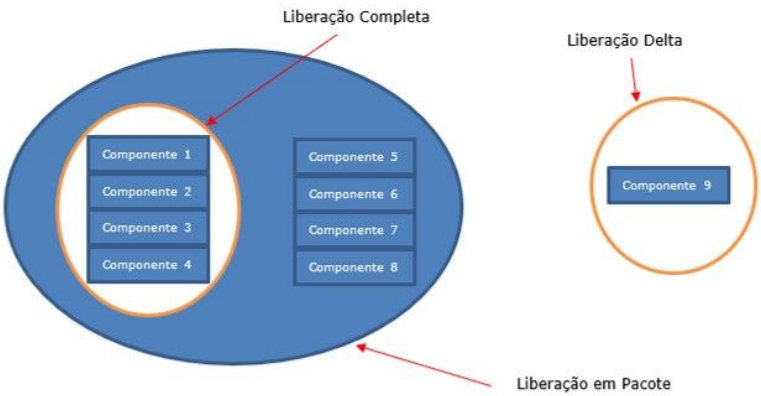
> ⚠
>
> ### Atenção
>
> Liberações emergenciais não devem ser banalizadas, devendo ser usadas com parcimônia, apenas como último recurso. Mudanças emergenciais alteram o versionamento de sua própria maneira, como por exemplo de 2.40 para 2.4.1.

Liberações baseadas no nível de agrupamento

Listaremos mais três tipos, que novamente podem implementar de uma a n mudanças por vez. A categorização agora se dá com base na forma como agrupam os itens de configuração sendo liberados.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Possível fluxo para mudança emergencial.

Essas liberações são:

Liberações Delta

Liberações que incluem apenas os Itens de configuração que foram de fato alterados, mesmo que façam parte de um grupo de itens de configuração que trabalham em conjunto. Um exemplo seria a liberação de uma única tela que foi alterada em um módulo de um sistema.

Liberações completas

Liberações que incluem todos os elementos de um grupo de itens de configuração que trabalham em conjunto, mesmo aqueles que não foram diretamente alterados. Um exemplo seria a liberação de um módulo de um sistema completo, mesmo que somente uma tela tenha sido alterada.

Liberações em pacote

Liberações que incluem várias liberações Delta e/ou completas. São agrupadas na estrutura de pacote por algum motivo, tais como interdependências, fazer melhor uso de recursos, cronogramas, prioridades do negócio etc.

Assista o vídeo e saiba mais sobre gerenciamento da construção e liberação.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Boas práticas no gerenciamento de liberação

O gerenciamento de liberação coordena a distribuição de itens de configuração fundamentais para os projetos, operações e para as organizações, certificando-se que essas liberações estão alinhadas com as prioridades do negócio.

Ao fazer isso, ajuda na identificação e gestão de riscos, na coordenação de recursos, na aderência a regulações e recomendações vindas de auditorias e mantém o desenvolvimento de software em linha com as necessidades dos usuários.

Para que tudo isso seja atingido, uma série de boas práticas existe e pode ser adotada conforme faça sentido. Entre essas práticas podemos citar:

Instituir um gerente de liberações com papel bem definido

Embora essa seja uma prática consolidada em grandes organizações, não é uma prática tão difundida em organizações menores, sendo que outros profissionais, tais como os gerentes de projeto ou gerentes de mudança, acabam por assumir a função de gerente de liberação. Isso ocorre, dentre outros motivos:

- A organização não ter recursos suficientes para manter um gerente de liberação;
- Por se acreditar que não há volume de trabalho para justificar a existência de um gerente de liberação em tempo integral;
- Pelo fato de o papel de gerente de liberação não ter sua importância reconhecida pela organização.

Independentemente do caso, é interessante que ao menos exista o papel bem definido de gerente de liberação, com responsabilidades claras, autoridade formal e definida e escopo de atuação difundido em toda a organização, mesmo que esse papel seja acumulado com outros.

Uma forma de endereçar todas essas considerações é estabelecer o rodízio dos gerentes de projeto na função de gerente de liberação.

Durante o mandato como gerente de liberação, o gerente de projeto ocuparia os dois cargos de maneira part-time. Assim sendo, não só se garante a disseminação do conhecimento acerca da liberação, mas também se assegura um mínimo nível de apoio.

Estabelecer um ciclo regular para as liberações

Esse é um começo importante, pois estabelece uma linha de tempo clara a todas as partes interessadas, desde a engenharia até os clientes e usuários. Isso estimula esses usuários a se comprometerem com a definição dos requisitos em tempo hábil, pois sabem que a obtenção de correções e novas funcionalidades estão atreladas a um cronograma pré-acordado.

Isso é valioso, em especial para organizações que ainda não são ágeis. Ao mesmo tempo, essa prática dá visibilidade ao time de engenharia sobre as expectativas que virão, otimizando o uso dos recursos, bem como gerando o comprometimento com as entregas, desde que atreladas ao ciclo acordado.

- Considerar um ciclo ágil para as liberações: Com a popularização dos *frameworks* ágeis, como, por exemplo, Scrum e XP, é natural que surjam esforços para tornar as liberações igualmente ágeis. Nessas abordagens, é comum que o comitê de mudanças seja descentralizado, permitindo a facilitação do aceite e implementação de mudanças a cada iteração ágil.

Se as mudanças são flexibilizadas, a liberação também precisa ser, e uma das formas de se atingir esse objetivo é por meio do estabelecimento de um cronograma previsível de liberações atreladas ao próprio cronograma das iterações ágeis.

Como exemplo podemos citar o planejamento de liberações para ocorrerem a cada 12 semanas de iterações de desenvolvimento, o que normalmente irá corresponder a 6 iterações de 2 semanas, ou 4 iterações de 3 semanas, ou 3 iterações de 4 semanas.

Informações sobre variantes ou ramos de um item de configuração também podem ser usadas para determinar se uma mudança tem impactos em vários locais.

Ainda é possível usar metadados para determinar se algum item de configuração possui solicitações de mudança em aberto, tais como mudanças em processo de implementação ou mesmo aguardando a decisão final sobre se serão ou não implementadas.

Incorporar requisitos claros com critérios de aceitação ainda mais claros

Embora essa não seja uma prática diretamente ligada à gestão de liberação, é importante que os profissionais envolvidos nas liberações sejam campeões suportando essa prática da melhor maneira possível.

Isso porque o gerenciamento da liberação funciona a pleno vapor quando se tem visibilidade muito clara e não-ambígua sobre quando os itens de configuração estão realmente prontos para serem entregues.

Ao se operacionalizar essa prática, evita-se ainda possíveis conflitos entre a gestão da liberação e a gestão de projetos, gestão de testes e outros.

Estabelecer uma infraestrutura de liberação o mais rapidamente possível

---

Qualquer coisa necessária para a emissão do *software* / documentação e para que os usuários possam usá-los faz parte da infraestrutura de liberação. Só construir o software não é suficiente para garantir satisfação, é necessário assegurar que o acesso e uso ao software também ocorrerá da forma o mais corriqueira possível.

Para que isso ocorra, a determinação do que exatamente faz parte da infraestrutura de liberação precisa ser feita o mais cedo possível, antes mesmo que o time de engenharia finalize o produto. Podem estar inclusos elementos de hardware, software, redes, permissões de acesso, entre outros.

Considerar todos esses elementos ajuda a eliminar gargalos e surpresas desagradáveis que podem minar todo o esforço de desenvolvimento.

Assista o vídeo e aprenda boas práticas no gerenciamento de liberação.

Conteúdo interativo

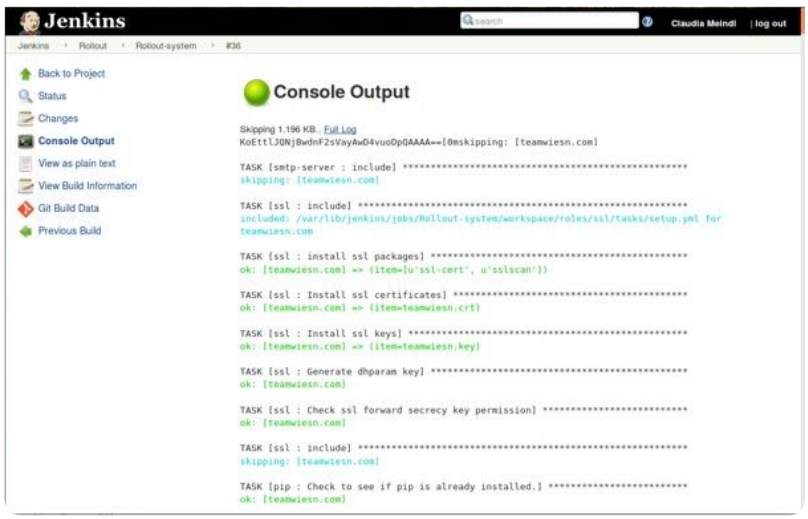
Acesse a versão digital para assistir ao vídeo.

A ferramenta Jenkins

A ferramenta Jenkins, originalmente conhecida como Hudson, é uma ferramenta opensource muito conhecida e usada no mercado por suas capacidades voltadas para a Integração Contínua e Gestão de Construção. Suporta esforços de desenvolvimento de software com base em diversas tecnologias, algumas mais legadas, tais como Java e PHP, e outras mais recentes, tais como Ruby on Rails e Groovy.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Visão do console da ferramenta Jenkins.

Apesar dos usos específicos previamente mencionados da ferramenta Jenkins, ela é flexível o bastante para ser usada como ferramenta de gestão de liberação. É ainda flexível para rodar como uma aplicação isolada (standalone) ou como um servidor de aplicação Java da mesma forma que é possível para o Apache Tomcat, por exemplo.

Saiba mais

O Jenkins, para ser instalado, exige portanto a presença do Java na máquina hospedeira, no mínimo na versão 5, sendo recomendada a versão 5.0 do Java Development Kit (JDK).O Jenkins encontra-se disponível para Windows, Linux e Mac OS X e se integra bem com outras ferramentas, tais como Controle de Versão (Git e SVN, por exemplo).

Muito importantes nesse processo são as ferramentas de controle de ativos, tais como JFrog, e de gestão de construção, tais como o Maven, pois dão subsídio para que o Jenkins funcione como uma ferramenta de Gestão de Liberação, promovendo, de maneira transparente, versões intermediárias para versões definitivas, oficializando assim as suas liberações.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Visão da página inicial do Jenkins.

Exemplo

É possível usar os metadados providos pelo JFrog, e então usar o Jenkins para controlar liberações em diferentes ambientes, tais como ambientes de produção, em de teste de qualidade, de teste de homologação etc.

Assim se mantém a rastreabilidade automatizada de quais versões de componentes são liberadas em conjunto para cada aplicação e para cada liberação, bem como quais componentes foram testados e receberam as aprovações devidas etc.

Infelizmente existem também desvantagens no uso do Jenkins. Veja a seguir.

- Por exemplo, a ferramenta é notoriamente pobre com relação a métricas, falhando em dar visibilidade a mudanças-chave na organização ou a desafios enfrentados por funcionários.

- Não é possível por exemplo identificar os funcionários mais produtivos, as mudanças mais recentes que causaram maior impacto positivo ou negativo na organização etc.

- Outro ponto negativo, de certa forma atrelado ao anterior, é a dificuldade que clientes do Jenkins têm com a rastreabilidade e contabilidade de mudanças feitas por diversos membros dos times de desenvolvimento. Isso porque a maioria das liberações e construções via Jenkins são feitas por um único usuário comum, tornando impossível o bom rastreamento das liberações. Toda a rastreabilidade precisa, portanto, ser transferida para outras ferramentas, tais como Git, e ser executada a nível de código-fonte.

- Mais um ponto de desvantagem é a dificuldade na colaboração, pois, por meio do Jenkins, não é possível para um desenvolvedor saber quais foram as mudanças realizadas por outros membros da equipe.

- A ferramenta também é um tanto inflexível quanto a customizações, não permitindo por exemplo a adaptação a coisas como fluxos de trabalho, reporte, gestão de mudanças voltadas para endereçamento de questões de segurança etc.

Assista o vídeo e aprenda sobre a ferramenta de Jenkis.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Verificando o aprendizado

Questão 1

Uma organização provedora de software de contabilidade, e que começou apenas com dois funcionários, cresceu, recentemente atingindo 250 colaboradores que atendem a algumas dezenas de clientes diferentes. A alta administração deseja, portanto, abandonar o método de trabalho ad-hoc atual, passando a adotar boas práticas reconhecidas no mercado.

Para tal, contratou você como consultor externo, pedindo que foque seus esforços iniciais em melhorar as liberações, pois há uma tendência alarmante em se introduzir problemas em produção a cada emissão de novos produtos.

Como consultor, você foca inicialmente não necessariamente nas práticas de liberação em si, mas sim nas práticas de

A Mudança.

B Configuração.

C Construção.

D Testes.

E Integração contínua.

A alternativa D está correta.

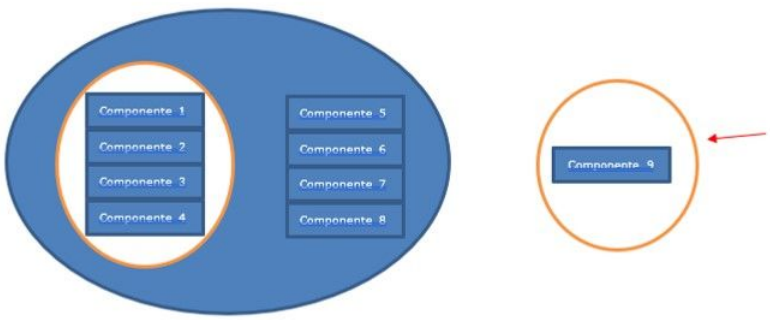
A ligação entre as liberações e as práticas de teste é motivo de destaque no gerenciamento de liberações. Embora todos os outros pontos sejam importantes, se existem problemas com as liberações, um dos primeiros pontos a serem checados como fonte destes problemas normalmente serão as práticas de teste relacionadas.

A razão disso é simples: garantir visibilidade e acordos claros com relação a quando um item de configuração e / ou produto estão efetivamente prontos para serem liberados.

Questão 2

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



A Liberação Delta.

B Liberação menor.

C Liberação em pacote.

D Liberação emergencial.

E Liberação maior.

A alternativa A está correta.

Liberações Delta incluem apenas os Itens de configuração que foram de fato alterados, mesmo que façam parte de um grupo de itens de configuração que trabalham em conjunto. Um exemplo seria a liberação de uma única tela que foi alterada em um módulo de um sistema.

Questão 3

Endereçam consertos rápidos para problemas inesperados. Estamos falando de liberações do tipo:

A Padrão.

B Normal.

C Delta.

D Completa.

E Emergencial.

A alternativa E está correta.

Liberações emergenciais endereçam consertos rápidos para problemas inesperados e que trazem níveis de impactos altos ou inaceitáveis para as organizações e negócios.

Liberações emergenciais se destinam também a implementar soluções preventivas para problemas que ainda nem ocorreram, mas que podem ocorrer a qualquer momento, gerando impactos altos demais para que liberações maiores ou menores agendadas possam ser executadas.

Questão 4

Implementar um comitê de mudanças descentralizado:

A É uma prática não-recomendada, pois pode gerar muitos efeitos adversos no controle de mudanças e na gestão de liberação.

B É uma prática não-recomendada, pois não existe algo como controle de mudanças ou gerenciamento de liberação ágil.

C Não faz diferença, já que o comitê de mudanças é apenas um dos vários papéis envolvidos com mudanças e liberações.

D É uma prática recomendada para toda e qualquer organização, pois é importante que se mantenha sintonia com tendências atuais.

E É uma prática recomendada para os casos em que uma organização deseja dar passos rumo à agilidade e apresenta condições para isso, tal como apoio da alta administração.

A alternativa E está correta.

Embora práticas ágeis sejam uma tendência na área de tecnologia da informação há vários anos, não é qualquer organização que está preparada e capacitada para adotá-las. É preciso analisar o contexto, entender o nível de apoio vindo da alta administração e até mesmo das próprias equipes envolvidas com as mudanças e liberações, e ainda, a real necessidade de se adotar práticas ágeis.

Comitês de mudança descentralizados são apenas uma ação que ajuda na obtenção de agilidade, e, dentro do exposto, deve ser adotada somente como parte de um plano maior e mais abrangente.

Questão 5

A ferramenta Jenkins é primariamente uma ferramenta de:

A Controle de Mudança.

B Integração Contínua.

C Desenvolvimento de *software*.

D Gestão de Liberação.

E Teste de *Software*.

A alternativa B está correta.

A ferramenta Jenkins, anteriormente conhecida como Hudson, sempre teve como principal destino o uso voltado para práticas de integração contínua. Dentro de seu arcabouço de funcionalidades oferecidas, inclui também capacidades voltadas para a gestão de liberação.

Considerações finais

Neste conteúdo, estudamos dois processos essenciais no ciclo de vida do desenvolvimento de software: o gerenciamento de construção e o gerenciamento de liberação. Inicialmente, abordamos os fundamentos do gerenciamento de construção, destacando sua importância para garantir consistência na geração de artefatos e na gestão de dependências, reduzindo conflitos e promovendo maior controle sobre o ambiente de desenvolvimento.

Exploramos também boas práticas voltadas à automação da construção de software e conhecemos duas ferramentas amplamente utilizadas nesse contexto: Maven e Gradle. Ambas desempenham papel fundamental na automatização de tarefas como compilação, empacotamento e execução de testes, contribuindo para a padronização e eficiência dos processos.

Na sequência, voltamos nosso foco para o gerenciamento de liberação, tema central dentro da gerência de configuração. Apresentamos os conceitos fundamentais e os diferentes tipos de liberação, além de práticas recomendadas para garantir que versões sejam entregues de forma segura, previsível e controlada.

Por fim, destacamos o uso do Jenkins como ferramenta de apoio ao gerenciamento de liberação, evidenciando sua aplicação na orquestração de tarefas e no suporte a práticas de integração e entrega contínuas. A ferramenta permite automatizar etapas críticas, trazendo mais agilidade e confiabilidade ao processo de liberação de software.

Com isso, consolidamos uma compreensão abrangente sobre os mecanismos e ferramentas que viabilizam entregas mais estruturadas, automatizadas e eficazes, preparando você para aplicar esses conhecimentos em contextos profissionais que exigem controle e qualidade no desenvolvimento de software.

Explore +

- Visite o site DevMedia e leia sobre a automação do processo de construção.

- Pesquise a definição real do processo de Liberação e Implantação usada pelo Conselho de Justiça Federal.

Referências

- AIELLO, B. **Configuration Management Best Practices**. 1.ed. Pearson, 2013.

- BOURQUE, P.; FAIRLEY, R. **Software Engineering Body of Knowledge (SWEBOK)**. 3.ed. IEEE Computer Society, 2017.

- **Dataverse Project**. Consultado na internet em 1 fev. 2017.14 nov. 2020.

- HAAS, J. **Configuration Management Principles and Practice**. 1.ed. Addison Wesley, 2003.

- KLEIN, H. **Gradle Dependency Management**. 1.ed. PACKT Publishing, 2015.

- SMART, J. Jenkins **The Definitive Guide Continuous Integration for the Masses**. 1.ed. O'Reilly, 2011.

- SIRIWARDENA, P. **Maven Essentials Get Started With The Essentials of Apache Maven and you're your build your automation system up and running quickly**. 1.ed. PACKT Publishing, 2015. Acesso em 14 nov. 2020.